```
import numpy as np
from bisect import bisect_left
from IPython.display import display
import sympy as sp
from math import prod


x_sym = sp.Symbol('x')
x_star = 0.2
x = np.array([-0.2, 0, 0.2, 0.4, 0.6])
y = np.array([1.7722, 1.5708, 1.3694, 1.1593, 0.9273])
f = dict(zip(x, y))
eps = 1e-3
n = len(x)
```

Численное дифференцирование

Введём функцию для удобного вычисления разделённых разностей, которая будет использоваться при построении интерполяционного многочлена Ньютона. Кэширование результатов позволит избежать повторных вычислений для одних и тех же аргументов.

```
from functools import lru_cache


@lru_cache(maxsize=None)
def divided_diff(*x_args):
    if len(x_args) == 1:
        return f[x_args[0]]
    else:
        return (
            divided_diff(*x_args[1:])
            - divided_diff(*x_args[:-1])
        ) / (x_args[-1] - x_args[0])
```

Определим функции получения интерполяционного многочлена и произвдений $(x - x_i)$

```
def get_w(i, order, x_data):
    return prod(x_sym - xi for xi in x_data[i:i + order])


def get_interpolation_polynomial(i, order, x_data):
    phi = sum(
        get_w(i, k, x_data) * divided_diff(*x_data[i:i + k + 1])
        for k in range(order + 1)
    )
    return phi
```

Построим интерполяционные многочлены первой и второй степени для узлов, ближайших к x^*. Выберем индекс i так, чтобы $x[i] < x^* \leq x[i+1]$.

```
i = max(0, bisect_left(x, x_star) - 1)

phi1 = get_interpolation_polynomial(i, 1, x)
phi2 = get_interpolation_polynomial(i, 2, x)

display(sp.expand(phi1))
display(sp.expand(phi2))
```

$1.5708 - 1.007x$

$-0.108749999999999x^2 - 0.98525x + 1.5708$

Посмотрим на левостороннюю и правостороннюю производные в точке x^*, используя формулы для численного дифференцирования.

```
left_derivative = (y[i + 1] - y[i]) / (x[i + 1] - x[i])
right_derivative = (y[i + 2] - y[i + 1]) / (x[i + 2] - x[i + 1])

left_derivative, right_derivative
```

```
(np.float64(-1.0070000000000001), np.float64(-1.0504999999999998))
```

Теперь получим производную в точке x^*, используя интерполяционный многочлен первой степени.

```
phi1_derivative = float(sp.diff(phi1, x_sym).subs(x_sym, x_star).evalf())
phi1_derivative
```

```
-1.0070000000000001
```

Посчитаем вторую производную в точке x^*, используя формулу для численного дифференцирования второй производной

```
second_derivative = 2 * (
    (
        (
            (y[i + 2] - y[i + 1]) / (x[i + 2] - x[i + 1])
        ) - (
            (y[i + 1] - y[i]) / (x[i + 1] - x[i])
        )
    ) / (
        x[i + 2] - x[i]
    )
)
second_derivative
```

```
np.float64(-0.21749999999999825)
```

Посчитаем вторую производную в точке x^*, используя интерполяционный многочлен второй степени.

```
phi2_second_derivative = float(sp.diff(phi2, x_sym, 2).subs(x_sym, x_star).evalf())
```

```
phi2_second_derivative
```

```
-0.21749999999999825
```

Валидация

Сравним результаты численного дифференцирования и интерполяционного многочлена первой степени для первой производной в точке x^*.

```
np.isclose(phi1_derivative, left_derivative, eps)
```

```
np.True_
```

Сравним результаты численного дифференцирования и интерполяционного многочлена второй степени для второй производной в точке x^*.

```
np.isclose(phi2_second_derivative, second_derivative, eps)
```

```
np.True_
```